Many-Task Federated Fine-Tuning via Unified Task Vectors

Vasileios Tsouvalas[1] , **Tanir Ozcelebi** and **Nirvana Meratnia**[1]
[1]Eindhoven University of Technology
{v.tsouvalas, t.ozcelebi, n.meratnia}@tue.nl

arXiv:2502.06376v3 [cs.LG] 8 Jul 2025

Abstract

Federated Learning (FL) traditionally assumes homogeneous client tasks; however, in real-world scenarios, clients often specialize in diverse tasks, introducing *task heterogeneity*. To address this challenge, Many-Task FL (MaT-FL) has emerged, enabling clients to collaborate effectively despite task diversity. Existing MaT-FL approaches rely on client grouping or personalized layers, requiring the server to manage individual models and failing to account for clients handling multiple tasks. We propose `MaTU`, a MaT-FL approach that enables joint learning of task vectors across clients, eliminating the need for clustering or client-specific weight storage at the server. `MaTU` constructs a single "*unified*" task vector per client and leverages lightweight task-specific modulators — binary masks and scalar rescalers — to disentangle and adapt per-task behavior. This design ensures stateless server aggregation and naturally scales to large task sets with minimal communication overhead. Evaluated across 30 datasets, `MaTU` achieves superior performance over state-of-the-art MaT-FL approaches, with results comparable to per-task fine-tuning, while delivering significant communication savings.

1 Introduction

Federated Learning (FL) has emerged as a collaborative learning paradigm, enabling joint training of neural network models across edge devices (refereed to as clients), while keeping data localized [McMahan *et al.*, 2017]. Traditional FL settings assume clients work on the same set of tasks; yet, in real-world cross-silo FL the inherent task heterogeneity among clients can naturally occur as clients typically specialize in specific tasks. This phenomenon, known as *task heterogeneity*, represents a novel and under-explored form of heterogeneity in FL, adding complexity to the learning process.

Recent studies [Chen *et al.*, 2023; Zhuang *et al.*, 2023; Cai *et al.*, 2023] have underscored the significance of *task heterogeneity* in FL. To address this diversity, traditional FL has evolved into Many-Task Federated Learning (MaT-FL) [Cai *et al.*, 2023] to enable clients to effectively collaborate, despite specializing in different tasks. It is worth to mention that while "*many-task*" has been interchangeably used as "*multi-task*" in conventional ML, early Multi-Task Federated Learning (MTFL) mainly addressed personalized FL, a subset of MaT-FL. MaT-FL is thus used to distinguish it from prior MTFL works [Cai *et al.*, 2023; Muhamed *et al.*, 2024; Zhuang *et al.*, 2023]. Current MaT-FL approaches address *task heterogeneity* by focusing on client grouping or split FL. For instance, FedBone [Chen *et al.*, 2023] employs split FL whereas the server needs to sustain a unique model per client. Conversely, MaT-FL [Cai *et al.*, 2023] and MAS [Zhuang *et al.*, 2023] concentrate on dynamically grouping client models based on task similarity, aggregating models among clients that handle similar tasks. Nevertheless, these strategies require the server to manage individual models for each client or task and fail to address scenarios where clients hold more than one task (e.g., a personal health tracker monitoring multiple biomarkers such as heart-rate, blood-pressure, blood glucose level, body temperature simultaneously). Training a single model among tasks offers considerable benefits in federated settings. Training individual models for each task is resource-intensive, whereas a unified model not only conserves resources but also potentially enhances performance by leveraging learning from diverse auxiliary tasks.

Concurrently, the advent of large-scale deep models trained on massive amounts of data in a self-supervised fashion has ushered in a new era of Foundation Models (FMs). These models, exemplified by Large Language Models (LLMs) and vision Foundation Models (vFMs) hold the promise of adapting to a wide range of downstream tasks. Furthermore, Parameter Efficient Fine-Tuning (PEFT) techniques, allowing FMs to reach performance levels comparable with fully fine-tuned models with only minimal adjustments to their parameters, have recently seen immense popularity, with LoRA adapters standing out for their efficacy and broad applicability in FL [Yi *et al.*, 2023; Cho *et al.*, 2023; Nguyen *et al.*, 2024; Ping *et al.*, 2024].

This paper addresses the challenge of adapting pretrained Foundation Models (FMs) to the MaT-FL setting, where clients hold an arbitrary (>1) number of tasks, and we aim to train a unified model across all tasks. We propose **Ma**ny-**T**ask FL via **U**nified Task Vectors (`MaTU`), a novel approach that eliminates the need for clustering or client-specific weight

storage at the server, enabling natural knowledge transfer across tasks in FL. Inspired by recent advancements in model merging and Task Arithmetic (TA) [Ilharco *et al.*, 2023; Yadav *et al.*, 2023a], `MaTU` introduces a task vector aggregation mechanism tailored for MaT-FL. Specifically, we determine task similarity directly based on the direction of clients' task vectors and constructs a "*unified*" task vector that encapsulates all tasks. To handle task-specific requirements, we augment the "*unified*" vector with lightweight modulators (i.e., binary task-specific masks and scalers) that facilitate knowledge transfer between related tasks while promoting weight disentanglement for dissimilar ones, ensuring robust performance across heterogeneous tasks in FL. By transmitting only a single vector and lightweight modulators per client, `MaTU` avoids the per-task overhead of existing MaT-FL methods, enabling both communication efficiency and scalability to large task sets. Concisely, our main contributions are as follows:

- We introduce `MaTU`, a MaT-FL approach enabling joint training across clients which holding multiple tasks without requiring server-side clustering or client-specific weight storage.
- We propose a novel task vector aggregation mechanism that constructs a unified task vector across tasks, while leveraging lightweight task-specific modulators to promote weight disentanglement and knowledge transfer.
- We conduct a comprehensive performance evaluation across 30 datasets, where `MaTU` outperforms state-of-the-art MaT-FL regimes with performance comparable to per-task adaptation fine-tuning, all while delivering significant communication savings.

2 Related Work

Federated PEFT. Parameter Efficient Fine-Tuning (PEFT) has emerged as an alternative fine-tuning strategy, where most of the pre-trained model parameters are frozen, and only a small portion of task-specific parameters are trained. Among the various PEFT methods, the injection of additional adapter modules, fine-tuned independently from the pre-trained model — such as Low-Rank Adaptation (LoRA)[Hu *et al.*, 2021] — has achieved state-of-the-art results across many large pre-trained models.

In the context of FL, PEFT-based approaches have demonstrated success in enhancing privacy[Sun *et al.*, 2024], communication [Tsouvalas *et al.*, 2023], and computation [Babakniya *et al.*, 2023] efficiency. SLoRA [Babakniya *et al.*, 2023] addresses data heterogeneity in federated settings by utilizing multiple LoRAs, while FFA-LoRA [Sun *et al.*, 2024] enhances privacy in FL by applying differential privacy techniques to LoRA modules. Moreover, DeltaMask [Tsouvalas *et al.*, 2023] combines LoRAs with probabilistic encoding to achieve significant bitrate reductions during FMs fine-tuning in FL, and FS-LLM [Kuang *et al.*, 2023] extends PEFT to LLMs in the federated settings. Nonetheless, most PEFT-based FL approaches primarily focus on single-task settings, overlooking their potential in many-task settings. In our work, we explore combining LoRAs with model merging techniques, such as TIES [Yadav *et al.*, 2023b], to address *task heterogeneity* in FL, enabling the training of a unified model across multiple tasks while achieving significant reductions in communication and computation overhead.

MaT-FL. Many-Task Federated Learning (MaT-FL) has emerged to tackle *task heterogeneity* in FL. From MTFL approaches, FedProx [Li *et al.*, 2020], introduced a proximity term to limit client updates from deviating too far from the global model in MTFL. Alternatively, FedBone [Chen *et al.*, 2023] used split FL to introduce task-specific personalized layers to handle multiple tasks in FL. Recent MaT-FL approaches primarily address *task heterogeneity* by focusing on client grouping [Cai *et al.*, 2023; Zhuang *et al.*, 2023; Lu *et al.*, 2023]. MaT-FL [Cai *et al.*, 2023] and MAS [Zhuang *et al.*, 2023] dynamically group clients based on task similarity, enabling model aggregation among clients with similar tasks. FedHCA [Lu *et al.*, 2023] extends this by supporting a variable number of tasks across clients, creating a more flexible framework. More recently, NTKFedAvg [Muhamed *et al.*, 2024] introduced task arithmetic in MaT-FL, enabling clients to train task-specific adapters and leverage server-side adapter fusion for optimizing multiple tasks. To further improve task disentanglement during model aggregation, they applied Neural Tangent Kernel (NTK) linearization over the model prior to training. Nevertheless, group-based MaT-FL approaches require the server to manage multiple models for each task group, assume task relationships are known in advance, and introduce significant client-side overhead, making scalability a challenge as the number of tasks increases. Training a single model across tasks offers considerable benefits: it reduces resource consumption and can enhance performance by leveraging diverse auxiliary tasks. While NTKFedAvg attempts to address these challenges, it focuses on a single task per client, whereas in practice, clients may hold multiple tasks. `MaTU` bridges this gap by training a unified model across all clients and tasks, dynamically building task correlations to enable knowledge transfer, and using task vector aggregation based on task similarity, removing the need for multiple server-side models.

3 Methodology

3.1 Preliminaries

Task Arithmetic. We denote with $\theta_p \in \mathbb{R}^d$ the pre-trained model weights of a FM. A task vector τ_t for task t is defined as $\tau_t = \theta_t^* - \theta_p$, where θ_t^* refers to the fine-tuned model weights of task t. Essentially, τ_t can be seen as a representation of the direction (*sign*) and amount (*magnitude*) of movement relative to θ_p needed in the d-dimensional weights space to lead to a low loss region for the task t. Additionally, a scalar multiplication with λ over τ_t, i.e., $\tau_{new}^t = \lambda \cdot \tau_t$ yields models parameters along the training trajectory, with positive λ values denoting task learning and negative λ values indicating task negation (unlearning) [Ortiz-Jimenez *et al.*, 2023]. Hence, adding K task vectors, a many-task ("*merged*") model can be obtained aiming to solve all K tasks as follows:

$$\theta_{\mathrm{m}} = \theta_p + \sum_{k=1}^{K} \lambda_k \cdot \tau_k \tag{1}$$

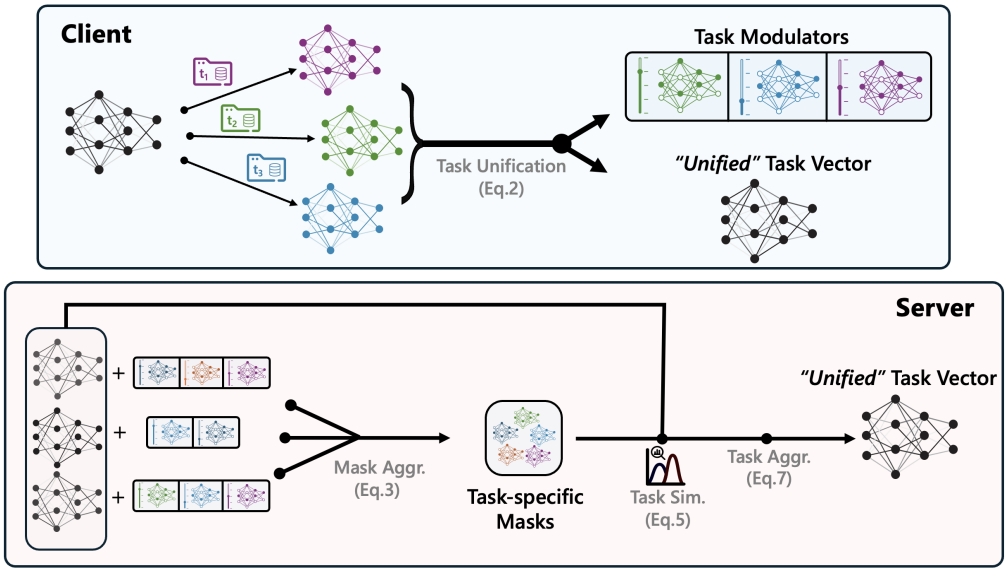


Figure 1: MaTU's training process: Clients update their task vectors, and transmit their "*unified*" task vector and task modulators to server, which constructs task masks, estimates task similarity via sign conflicts, and construct clients' task vectors using the top-κ similar tasks.

By only adjusting λ_k, the performance across K tasks can be improved. Building on the observations from [Ortiz-Jimenez *et al.*, 2023], TIES [Yadav *et al.*, 2023a] demonstrated that resolving sign conflicts before model merging enhances weight disentanglement (i.e., separating the influence of different task vectors), leading to improved task performance. Furthermore, re-scaling task vectors often yields better results than using unmodified vectors [Yang *et al.*, 2024; Yu *et al.*, 2024].

Unified Task Vector. Given a number of task vectors derived from a set of tasks, $t \in \mathcal{T}$, the "*task unification*" [Huang *et al.*, 2024] process begins by computing the aggregated sign vector across all task vectors, $\sigma = \text{sgn}\left(\sum_{i=1}^{|\mathcal{T}|} \tau^i\right)$. Next, the "*unified*" task vector is as follows:

$$\tau = \sigma \odot \mu \,, \tag{2}$$

where μ refers to the magnitude vector constructed by extracting the maximum absolute value from the task vectors whose signs align with σ. This electing procedure has shown to reserve the maximum amplitude and sign information shared by the task vectors, thereby maximally reducing interference [Huang *et al.*, 2024].

Weight disentanglement. Weight disentanglement — the model's ability to isolate the influence of different tasks on its weights — serves as a necessary condition for task arithmetic operations, and is an emerging property of pre-training [Ortiz-Jimenez *et al.*, 2023]. Since the sign of each weight indicates the direction that minimizes loss for a given task [Ortiz-Jimenez *et al.*, 2023; Yadav *et al.*, 2023a; Muhamed *et al.*, 2024], resolving sign conflicts among task vectors can promote weight disentanglement, leading to improved individual task performance [Yadav *et al.*, 2023a]. Therefore, in the context of the same pre-trained model, a high count of sign conflicts between two task vectors implies opposing influences on model weights, suggesting potential interference in their joint training.

3.2 MaTU :Efficient MaT-FL with Task Arithmetic.

Overview. Here, we present the general MaTU training pipeline (see Fig. 1). Clients initialize the same model f, parameterized by the pre-trained weights θ_p, and train on their local datasets D_n^t to generate individual task vectors τ_n^t, which are combined into a unified task vector τ_n. To account for the changes from τ_n^t to τ_n, clients create lightweight modulators — binary masks m_n^i and scaling values λ_n^i — which are transmitted along with τ_n. Upon receiving client updates, for each task, the server computes an average task mask, $\hat{m}^t$, to capture key areas in the task vectors, then performs task-specific aggregation to combine updates from clients sharing the task, followed by cross-task aggregation to enable knowledge transfer between related tasks. By averaging these two, the server creates the updated task vectors to compute unified task vectors and modulators for each client based on their assigned tasks, which then transmits to client. Next, clients use these modulators to adjust the unified task vector and derive the updated task vectors, marking the next federated round.

Notation. We use the following notation in the rest of the paper. Let $\mathcal{N}$ refer to a set of clients, each assigned a set of k tasks $\mathcal{T}_n$ ($k_n = |\mathcal{T}_n|$), which may overlap across clients (i.e., the unique set of tasks is given by $\mathcal{T} = \bigcup_{n=1}^{|\mathcal{N}|} \mathcal{T}_n$). Each task $t \in \mathcal{T}_n$ is associated with a locally stored dataset D_n^t, containing $|D_n^t|$ samples. $A \in \{0,1\}^{N \times T}$ denotes a binary matrix representing *task-to-client allocations*, where $A(n,t)$=1 if client n holds task t, and 0 otherwise. The neural network is denoted by f and parameterized by weights θ, where θ_p represents its pre-trained weights, and θ_t^* denotes its optimized weights after local training on task t. For each task t, its task

vector is given by $\tau^t = \theta_t^* - \theta_p$, where $\sigma^t = \text{sgn}(\tau^t)$ denotes the sign vector of τ_t, indicating the direction of movement (positive or negative) relative to θ_p.

Local Training with many-tasks. In a given round $r \in R$, the n-th client trains individually across the set of k_n locally stored tasks, and derives a set of local task vectors, one for each task $t \in \mathcal{T}_n$. Next, the "*task unification*" process is performed to derive the client's "*unified*" task vector, $\tau_n = \sigma_n \odot \mu_n$.

As a single task vector cannot capture all task-specific weights, leading to performance degradation, we introduce *lightweight* task-specific modulators that refine τ_n to approximate task-specific vectors, as in [Huang *et al.*, 2024]. Specifically, we construct a set of task-specific masks, $\mathcal{M}_n = \{m_n^i\}_{i=1}^{k_n}$, where each mask is defined as $m_n^i = (\tau_n^i \odot \tau_n > 0)$ for each task $t \in \mathcal{T}_n$; thus, aligning the direction of τ_n to each task vector. Similarly, to account for the magnitude shift between τ_n and individual task vectors, we introduce task-specific scaling parameters $\lambda_n = \{\lambda_n^i\}_{i=1}^{k_n}$, where $\lambda^i = \frac{\sum |\tau_n^i|}{\sum |m_n^i \odot \tau_n|}$.

Many-tasks Aggregation. Once the local training at round r is completed, the server holds each client's "*unified*" task vector and task-specific modulators (i.e., masks and re-scaling parameters). We aim to aggregate task vectors among clients holding a given task, while enabling *dynamic* knowledge transfer across similar tasks. Thus, we propose a simple-yet-effective *many-tasks aggregation* scheme tailored for MaT-FL, incorporating both task-specific and cross-task aggregation mechanisms.

Task-specific Aggregation. For each task t, we compute the average task-specific mask $\hat{m}^t$ across the clients that hold this task, defined as $\mathcal{N}^t = \{n \in \mathcal{N} \mid A(n,t) = 1\}$, by performing the following element-wise operation on its elements:

$$[\hat{m}^t]_j = \begin{cases} 1 & \text{if } \alpha_j^t \geq \rho \\ \alpha_j^t & \text{otherwise} \end{cases}, \text{ where } \alpha_j^t = \left| \frac{1}{|\mathcal{N}^t|} \sum_{n \in \mathcal{N}^t} \text{sgn}(m_n^t \odot \tau_n) \right| \quad (3)$$

where $[\hat{m}^t]_j$ represents the j-th element of $\hat{m}^t$, and ρ[1] is a threshold that determines the significance of each parameter for task t.

As demonstrated in [Tenison *et al.*, 2023], the agreement score α is closely related to client heterogeneity, with highly heterogeneity among clients resulting in α close to 0. As clients optimize their task vectors across distinct sets of tasks, the impact on each task-specific vector entry varies due to unique interference from their remaining tasks, exacerbating their heterogeneity. By adjusting the magnitude of updates based on the agreement score α, we effectively reduce task interference, enhancing task separation; thus, promoting weight disentanglement during FL training stage. Using the $\hat{m}^t$, we then compute the average task-specific vectors across clients as:

$$\hat{\tau}^t = \sum_{n \in \mathcal{N}^t} \gamma_n^t \cdot \lambda_n^t \cdot \hat{m}^t \odot \tau_n^t \,, \quad (4)$$

where $\lambda_{n'}^t$ is the task-specific re-scaler for client n' and task t, $\hat{m}_t$ is the aggregated task-specific mask for task t (Eq. 3), and γ_n^t represents clients' weight in the aggregation based on their respective dataset size (i.e., $\gamma_n^t = |D_n^t| / \sum_{n \in \mathcal{N}^t} |D_n^t|$), similar to FedAvg [McMahan *et al.*, 2017]. In essence, $\hat{\tau}^t$ captures the task-specific information aggregated across clients in the FL process.

Cross-task Aggregation. Apart from capturing knowledge among clients holding task t, leveraging information from similar tasks can enhance task-specific performance, especially for data-scarce tasks [Muhamed *et al.*, 2024]. To identify redundancies that facilitate joint learning and enhance individual task performance, we first examine task similarity—the relationships among tasks [Zamir *et al.*, 2018]. Most task similarity metrics evaluate knowledge transfer benefits based on task-specific data [Gholami *et al.*, 2023; Bao *et al.*, 2022] or labels [Ding *et al.*, 2022]; yet, access to these resources is often limited in FL. Instead, as a high count of sign conflicts between task vectors for the same pre-trained model indicates potential interference in joint training [Yadav *et al.*, 2023a], we construct a task similarity matrix from sign conflicts among aggregated task-specific vectors. Specifically, the similarity between tasks t and t' is defined as follows:

$$S(t,t') = \frac{1}{2}\left(\frac{1}{d}\sum_{i=1}^{d}\left(\text{sgn}\left([\hat{\tau}^t]_i\right) \cdot \text{sgn}\left([\hat{\tau}^{t'}]_i\right)\right) + 1\right) \quad (5)$$

where d is the dimension of the task vectors. Note that $S \in [0,1]$, with a higher score indicating better alignment between tasks. Next, for a given task t, we derive the set of top-κ similar tasks, $\mathcal{Z}^t = [\{t' \in \mathcal{T} \mid S(t,t') > \epsilon\}]_{\leq \kappa}$[2], and use it to compute the average cross-task vectors as follows:

$$\tilde{\tau}^t = \sum_{t' \in \mathcal{Z}^t} S(t,t') \cdot \hat{m}^t \odot \hat{\tau}^{t'} \,, \quad (6)$$

Note that we use $\hat{m}^t$, rather than $\hat{m}^{t'}$, to adapt the t-th task's vector based on the aggregated task vectors of task t', effectively enabling knowledge transfer across tasks.

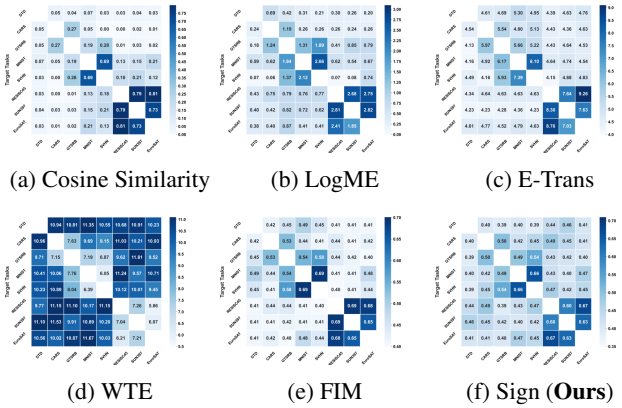
Forming clients updates. Given Eq. 4 and 6, we can compute the aggregated task-specific vector for task t on round r, as:

$$\tau^{t,r+1} = \hat{\tau}^{t,r} + \tilde{\tau}^{t,r} = \underbrace{\sum_{n \in \mathcal{N}^{t,r}} \gamma_n^t \cdot \lambda_n^{t,r} \cdot \hat{m}^{t,r} \odot \tau_n^{t,r}}_{\text{same-task}} + \underbrace{\sum_{t' \in \mathcal{Z}^{t,r}} S(t,t') \cdot \hat{m}^{t,r} \odot \hat{\tau}^{t',r}}_{\text{cross-tasks}} \quad (7)$$

For the set of tasks assigned to each clients, $\mathcal{T}_n$, we can now utilize the newly computed aggregated task-specific vector to derive their updated "*unified*" task vector, τ_n^{r+1}, and subsequently compute the set of task-specific masks, $\mathcal{M}_n^{r+1} = \{(\tau^{i,r+1} \odot \tau_n^{r+1} > 0)\}_{i=1}^{k_n}$, and scaling parameters $\lambda_n^{r+1} = \left\{\frac{\sum |\tau_n^{i,r+1}|}{\sum |m_n^{i,r+1} \odot \tau_n^{r+1}|}\right\}_{i=1}^{k_n}$. The client's "*unified*" task vector, τ_n^{r+1}, and task-specific modules, $\mathcal{M}_n^{r+1}$ and λ_n^{r+1}, are then transmitted to the client-side for subsequent training. To train task t, the client must modulate the "*unified*" vector as $\dot{\tau}_t^{r+1} = \lambda_n^{t,r+1} \cdot m_n^{t,r+1} \odot \tau_n^{r+1}$, and injects it to the pre-trained model (i.e., $\theta_t^{r+1} = \theta_p + \dot{\tau}_t^{r+1}$). In contrast to personalized

[1] $\rho = 0.4$ following [Tenison *et al.*, 2023]

[2] $\epsilon = 0.5$ filters out low-similarity tasks, and $[\cdot]_{\leq \kappa}$ refer to top-κ.



(a) Cosine Similarity (b) LogME (c) E-Trans

(d) WTE (e) FIM (f) Sign (**Ours**)

Figure 2: Comparison of sign vectors vs. state-of-the-art transferability estimation metrics across 8 datasets. In all metrics except WTE, higher values correspond to higher correlation.

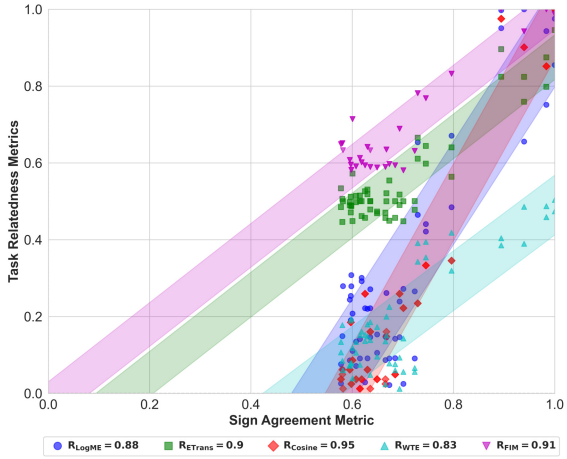


Figure 3: Pearson Correlation between sign-based and state-of-the-art task relatedness metrics across 8 datasets.

FL or group-based MaT-FL methods that require storing task-specific heads or group models, our design remains stateless — the server only handles task-level aggregation and does not retain client-specific weights across rounds.

4 Performance evaluation

Datasets. We evaluate `MaTU`'s performance across 30 vision classification datasets. Specifically, we consider 2 model merging benchmarks: (i) an 8-*task benchmark* comprising SUN397 [Xiao *et al.*, 2010], Cars [Krause *et al.*, 2013], RESISC45 [Cheng *et al.*, 2017], EuroSAT [Helber *et al.*, 2017], SVHN [Netzer *et al.*, 2011], GTSRB [Stallkamp *et al.*, 2011], MNIST [LeCun *et al.*, 2010], and DTD [Cimpoi *et al.*, 2014]; and (ii) a large-scale 30-*task benchmark* encompassing diverse vision tasks as proposed in [Huang *et al.*, 2024].

Baselines. We compare `MaTU` against centralized LoRA PEFT [Hu *et al.*, 2021], traditional FL methods (FedAvg [McMahan *et al.*, 2017], FedProx [Li *et al.*, 2020]), group-based approaches (*MaT-FL* [Cai *et al.*, 2023]), personalized FL approaches (FedPer [Arivazhagan *et al.*, 2019]) and related task-arithmetic methods like NTKFedAvg [Muhamed *et al.*, 2024]. In FedPer, the last ViT block and classifier are used as personalized layers. We compare all baselines in terms of model performance (i.e., accuracy on test set) and client's bitrate requirements (e.g. communicated bits per task in a round – bpr). All experiments utilize ViT-B/32 [Dosovitskiy *et al.*, 2021] pre-trained on ImageNet-21k [Deng *et al.*, 2009] and employ LoRA [Hu *et al.*, 2021] with a rank of 16 for PEFT, reporting test set accuracy.

FL Settings. We perform our FL simulations using Flower [Beutel *et al.*, 2020] with key parameters being number of clients (N), rounds (R), local epochs (E), clients' participation rate (ξ), number of tasks (T), class concentration across clients (ζ_c), and task concentration across clients (ζ_t). We used a Dirichlet distribution, denoted as Dir(α), for both data and task splitting, where α represents the concentration parameter, following [Li *et al.*, 2021]. We simulate non-IID settings with α=0.1 (ζ_c=ζ_t=0.1) and train federated models for 100 rounds (R=100) with 30 clients (N=30), each performing one local training step per round (E=1). Note that for ξ<1, clients are randomly selected.

Task Relatedness. We investigate the effectiveness of sign agreement among task vectors as a metric for measuring task similarity across datasets. To evaluate this, we conduct experiments with 8 vision classification tasks, comparing our sign-conflict approach against well-established task similarity metrics, including cosine similarity of weights [Vu *et al.*, 2022], E-Trans [Gholami *et al.*, 2023], FIM [Amari, 1998], and WTE distance [Liu *et al.*, 2025]. As shown in Fig.2, our sign-conflict method effectively identifies similar task groups among the datasets. Furthermore, Fig.3 highlights a strong Pearson correlation (>0.8) between the sign-conflict-based and all other task similarity metrics.

4.1 Results

Single-task Clients. We explore a "*simplified*" version of MaT-FL, where each client holds a single task. Specifically, we perform experiments with 30 clients (N=30) under low client participation rate (ξ=0.2), ensuring no overlap among client's tasks (ζ_t=0.0) in the 8-task benchmark. We report our findings in Table 1, from which we observe that `MaTU` achieves the highest average accuracy (84.32%) among all baselines – less than a 6% gap to individual task fine-tuning performance. Traditional FL methods like FedAvg, NTK-FedAvg, and FedProx, with average accuracies of 64.60%, 65.13%, and 67.14%, respectively, struggle to merge task-specific updates effectively, even in single-task settings. The Neural Tangent Kernel (NTK) linearization provides minimal improvements over standard FedAvg in task disentanglement (less than 1%). While MaT-FL improves performance by mitigating task conflicts through cosine similarity-based grouping aggregation, it exhibits the slowest convergence among all baselines due to restricted client interactions. FedPer's task-specific personalization layers deliver strong performance (82.11%) in single-task settings – as they are explicitly tailored to individual tasks.

Table 1: Performance evaluation of ViT-B/32 models on 8 tasks in a ***single-task per client*** setting. Reported bitrates (bpt) are in millions. Federated parameters are set to N=30, E=1, ζ_t=0.0, ξ=0.2 and R=100 (except *MaT-FL* where R=300).

Methods	SUN397	Cars	RESISC45	EuroSAT	SVHN	GTSRB	MNIST	DTD	Avg. Acc	Bitrate (bpt - M) ↓
Individual (Centralized)	74.81	76.93	96.05	99.43	97.05	98.57	99.67	79.21	90.21	–
FedAvg [McMahan *et al.*, 2017]	62.84	60.77	71.08	72.58	68.72	65.06	65.47	50.28	64.60	6.32
NTK-FedAvg [Muhamed *et al.*, 2024]	63.08	59.22	71.98	72.37	67.34	66.13	67.04	54.97	65.13	6.32
FedProx [Li *et al.*, 2020]	66.58	63.64	74.95	76.26	70.02	62.86	67.65	55.16	67.14	6.32
FedPer [Arivazhagan *et al.*, 2019]	68.57	**69.39**	88.37	90.33	90.28	91.53	92.44	67.98	82.11	**5.36**
MaT-FL [Cai *et al.*, 2023]	67.02	65.19	85.13	82.68	85.90	80.16	87.08	60.12	76.66	6.32
MaTU (Ours)	**71.61**	69.17	**91.27**	**92.59**	**93.59**	**94.10**	**96.16**	**70.09**	**84.32**	6.32

Table 2: Performance evaluation of ViT-B/32 models on 8 tasks in a ***multiple-task per client*** setting. Reported bitrates (bpt) are in millions. Federated parameters are set to N=30, E=1, ζ_t=0.5, ξ=0.2 and R=100 (except *MaT-FL* where R=300).

Methods	SUN397	Cars	RESISC45	EuroSAT	SVHN	GTSRB	MNIST	DTD	Avg. Acc	Bitrate (bpt - M) ↓
Individual (Centralized)	74.81	76.93	96.05	99.43	97.05	98.57	99.67	79.21	90.21	–
FedAvg [McMahan *et al.*, 2017]	50.21	48.50	56.97	57.86	55.19	52.04	52.26	40.01	51.63	20.43
NTK-FedAvg [Muhamed *et al.*, 2024]	55.37	53.27	62.30	63.62	58.56	52.79	56.27	46.28	56.06	20.43
FedProx [Li *et al.*, 2020]	57.64	53.84	63.26	64.44	59.17	52.90	56.76	42.54	56.32	20.43
FedPer [Arivazhagan *et al.*, 2019]	41.29	38.26	57.01	61.20	57.06	55.06	36.32	40.74	48.37	17.16
MaT-FL [Cai *et al.*, 2023]	61.52	56.67	76.48	73.95	76.96	69.67	74.64	50.88	67.60	20.43
MaTU (Ours)	**66.40**	**63.46**	**86.36**	**87.26**	**88.15**	**88.51**	**91.08**	**64.69**	**79.47**	**8.04**

In contrary, MaTU outperforms FedPer by 2.21% on average, requiring only a marginal increase in bitrate. These results highlight MaTU's ability to mitigate task interference via task-specific masks in aggregation, ensuring task disentanglement and superior performance without added communication overhead compared to standard FL regimes.

Multiple-task Clients. We now explore a generalized MaT-FL scenario, where each client can hold an arbitrary ($>$1) set of tasks. For this, we perform experiments with 30 clients (N=30) in both 8-task and 30-task benchmarks. Here, task conflicts are expected to intensify both within and across clients, as no restrictions are placed on task assignments (i.e., task groups are formed randomly given ζ_t), allowing conflicting tasks to exist on a client. We report our results for both benchmarks in Table 2 and Fig. 4.

8-*task benchmark*: Table 2 demonstrates that MaTU maintains strong performance in *multiple-task* settings, achieving an average accuracy of 79.47% – a modest 5% drop from single-task settings – while outperforming all baselines and effectively minimizing task interference via masking. Traditional FL methods, such as FedAvg and FedProx, exhibit significant performance deficits, with average accuracies well below their single-task counterparts, underscoring their limitations in addressing *task heterogeneity*. Notably, NTK-FedAvg surpasses these baselines, likely benefiting from improved task disentanglement through NTK-based linearization. In stark contrast, FedPer experiences a severe performance decline (≈40%) when transitioning from *single* to *multiple-task* settings, reflecting its inability to manage intra-client *task heterogeneity* due to its reliance on personalization layers. This highlights that personalized FL approaches are not designed to address the broader challenges of *task heterogeneity*, underscoring that personalization represents a subset of the more general MaT-FL problem. Conversely, *MaT-FL* achieves higher performance by employing similarity-based grouping to mitigate task conflicts, reducing interactions among dissimilar clients and tasks, but still lags MaTU by 10%. We further analyze how *MaT-FL* and MaTU handle such conflicts in Fig.˜6a.

Beyond test accuracy, MaTU offers substantial communication and storage efficiency in *multiple-task* settings. As shown by the bpt column in Table 2, competing methods transmit multiple adapters — one per task—leading to high communication costs and server-side storage of client- or group-specific weights. In contrast, MaTU transmits only a single unified task vector per client, along with lightweight modulators (a binary mask and a scalar per task), enabling stateless aggregation and scalable deployment without storing any client-specific parameters.

30-*task benchmark*: We evaluate MaTU's scalability to 30 tasks and compare it to *MaT-FL*, the strongest-performing baseline in *multiple-task* settings (see Table 2). As shown in Fig.4, MaTU achieves an average normalized performance of 77.40%, significantly surpassing *MaT-FL*'s 52.62% across most datasets, except MangoLeafDB. This demonstrates MaTU's ability to minimize task interference through training and delivers performance comparable to individual task fine-tuning. An elaborate analysis of MaTU's effectiveness with respect of increase of client's tasks is presented in Fig. 5. More importantly, beyond superior joint training performance, MaTU constructs a "*unified*" task vector that encapsulates all tasks and can be swiftly adapted to individual tasks using computationally lightweight *modulators*, enabling significant storage savings during inference, particularly as model size scale.

Scaling clients tasks. We further analyze how MaTU handles an increasing number of tasks per client. Specifically, we conduct experiments on the 30-task benchmark, varying each client's task groups from 2 to 30, and report both (i) communication cost per round (in MB) and (ii) average normalized performance across tasks (vs individual task fine-tuning). As shown in Fig.5a, MaTU introduces minimal communication overhead as client tasks scale. This efficiency arises from transmitting a single adapter per client, regardless of number of tasks, supplemented by lightweight task-specific modulators (a binary mask

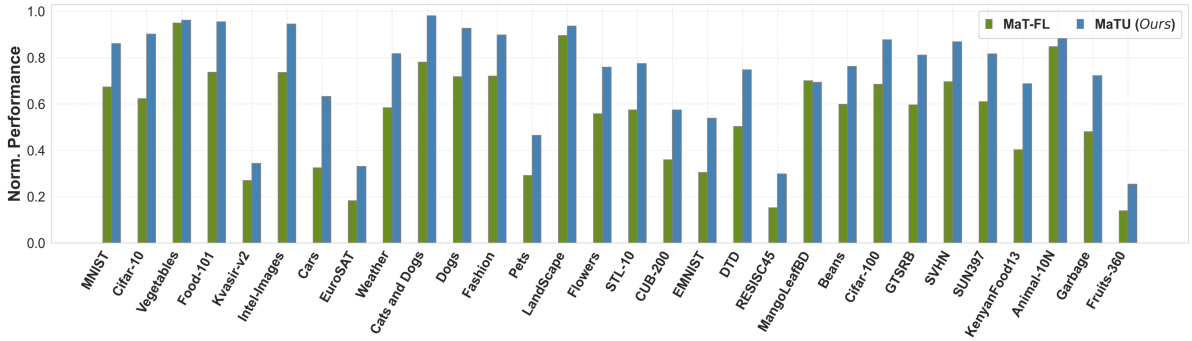
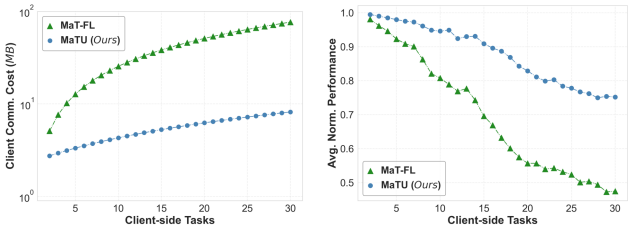
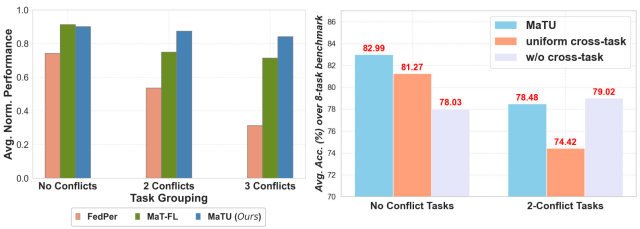


Figure 4: Performance evaluation of ViT-B/32 models on the 30-task benchmark. We report test set accuracy normalized to individual task fine-tuning performance. Federated parameters are set to N=30, R=300, E=1, ζ_t=0.2, and ξ=1.0.



(a) Comm. Overhead (b) Norm. Performance.

Figure 5: Impact of scaling number of tasks assigned to clients on MaTU. Performance evaluation of ViT-B/32 on the 30-task benchmark. We report (a) communication cost for 1 federated round, and (b) normalized test set accuracy vs individual task fine-tuning. Federated parameters are set to N=10, R=300, ξ=1 and E=1.



(a) Task conflict groups (b) Cross-task aggregation

Figure 6: Performance evaluation of ViT-B/32 on the 8-task benchmark under (a) task conflict groups and (b) cross-task aggregation variants. We report (a) normalized test set accuracy vs individual task fine-tuning and (b) average accuracy on test set. Federated parameters are set to N=10, R=300, ξ=1 and E=1.

and a scalar value), unlike the multiple adapters required by*MaT-FL* and other baselines. Furthermore, in terms of model performance, Fig.5b shows that *MaT-FL* suffers from a sharp performance drop when clients are assigned more than 5 tasks, highlighting its limitations in managing high *task heterogeneity*. In contrary, MaTU demonstrates to have a solid performance – even when clients train on over 15 tasks – highlighting its ability to effectively disentangle tasks in the weight space through task-specific modulators.

Highly Conflicting task groups. Here, we evaluate MaTU's performance in scenarios with highly conflicting tasks within client's task group. Using the 8-task benchmark, we focus on the 3 distinct task clusters identified in Fig.2. Specifically, we conduct experiments with 10 clients, each assigned a fixed group of three tasks, designed to include no, 2, or 3 highly dissimilar tasks, referred to as *no conflict*, *2-conflict*, and *3-conflict* task groups, respectively. This setup enables us to assess *how MaTU handles task conflicts on the clients* and compare its performance against baselines. As shown in Fig. 6a, MaTU achieves consistently high performance with minimal drop (less than 6%), unlike baselines such as FedPer, which struggles to train multi-task models even without conflicting task groups, and *MaT-FL*, which suffers from a more pronounced performance drop as task conflicts increase.

Cross-task aggregation. We also analyze the benefits of cross-task aggregation (Eq. 6) by comparing MaTU against two variants: one without cross-task aggregation and another using uniform cross-task averaging, where all task vectors are treated equally regardless of similarity. As shown in Fig.6b, MaTU achieves superior performance in the absence of conflicts and effectively mitigates negative transfer under *2-conflict* settings. In contrast, the uniform variant degrades under task heterogeneity, while removing cross-task aggregation entirely limits performance.

5 Conclusions

In this work, we introduced MaTU, a MaT-FL approach designed to address task heterogeneity by relying on task vectors that creates a "*unified*" task vector – encapsulating all tasks – together with lightweight task modulators. By eliminating the need for clustering or client-specific weight storage, MaTU enables scalable, low-overhead MaT-FL even with large numbers of tasks. Our evaluations across 30 datasets demonstrated that MaTU not only achieves superior performance compared to state-of-the-art MaT-FL regimes but also delivers results comparable to per-task fine-tuning while significantly reducing communication overhead; thus highlighting MaTU's potential under highly heterogeneous FL cases.

Acknowledgments

This work was funded by the DAIS project, which has received funding from ECSEL Joint Undertaking under grant agreement No 101007273, and the Bio-Curity project, which has received funding from Eureka Cluster Xecs under grant agreement No 2022016.

References

[Amari, 1998] Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10:251–276, 1998.

[Arivazhagan *et al.*, 2019] Manoj Ghuhan Arivazhagan, Vinay Aggarwal, Aaditya Kumar Singh, and Sunav Choudhary. Federated learning with personalization layers, 2019.

[Babakniya *et al.*, 2023] Sara Babakniya, Ahmed Roushdy Elkordy, Yahya H. Ezzeldin, Qingfeng Liu, Kee-Bong Song, Mostafa El-Khamy, and Salman Avestimehr. Slora: Federated parameter efficient fine-tuning of language models, 2023.

[Bao *et al.*, 2022] Yajie Bao, Yang Li, Shao-Lun Huang, Lin Zhang, Lizhong Zheng, Amir Zamir, and Leonidas Guibas. An information-theoretic approach to transferability in task transfer learning, 2022.

[Beutel *et al.*, 2020] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Hei Li Kwing, Titouan Parcollet, Pedro PB de Gusmão, and Nicholas D Lane. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.

[Cai *et al.*, 2023] Ruisi Cai, Xiaohan Chen, Shiwei Liu, Jayanth Srinivasa, Myungjin Lee, Ramana Kompella, and Zhangyang Wang. Many-task federated learning: A new problem setting and a simple baseline. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 5037–5045, 2023.

[Chen *et al.*, 2023] Yiqiang Chen, Teng Zhang, Xinlong Jiang, Qian Chen, Chenlong Gao, and Wuliang Huang. Fedbone: Towards large-scale federated multi-task learning. *arXiv preprint arXiv:2306.17465*, 2023.

[Cheng *et al.*, 2017] Gong Cheng, Junwei Han, and Xiaoqiang Lu. Remote sensing image scene classification: Benchmark and state of the art. *Proceedings of the IEEE*, 105(10):1865–1883, 2017.

[Cho *et al.*, 2023] Yae Jee Cho, Luyang Liu, Zheng Xu, Aldi Fahrezi, Matt Barnes, and Gauri Joshi. Heterogeneous loRA for federated fine-tuning of on-device foundation models. In *International Workshop on Federated Learning in the Age of Foundation Models in Conjunction with NeurIPS 2023*, 2023.

[Cimpoi *et al.*, 2014] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2014.

[Deng *et al.*, 2009] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

[Ding *et al.*, 2022] Nan Ding, Xi Chen, Tomer Levinboim, Beer Changpinyo, and Radu Soricut. Pactran: Pac-bayesian metrics for estimating the transferability of pretrained models to classification tasks, 2022.

[Dosovitskiy *et al.*, 2021] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.

[Gholami *et al.*, 2023] Mohsen Gholami, Mohammad Akbari, Xinglu Wang, Behnam Kamranian, and Yong Zhang. Etran: Energy-based transferability estimation, 2023.

[Helber *et al.*, 2017] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification, 2017.

[Hu *et al.*, 2021] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[Huang *et al.*, 2024] Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. Emr-merging: Tuning-free high-performance model merging, 2024.

[Ilharco *et al.*, 2023] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic, 2023.

[Krause *et al.*, 2013] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *2013 IEEE International Conference on Computer Vision Workshops*, pages 554–561, 2013.

[Kuang *et al.*, 2023] Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning, 2023.

[LeCun *et al.*, 2010] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, 2, 2010.

[Li *et al.*, 2020] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks, 2020.

[Li *et al.*, 2021] Qinbin Li, Yiqun Diao, Quan Chen, and Bingsheng He. Federated learning on non-iid data silos: An experimental study, 2021.

[Liu *et al.*, 2025] Xinran Liu, Yikun Bai, Yuzhe Lu, Andrea Soltoggio, and Soheil Kolouri. Wasserstein task embedding for measuring task similarities. *Neural Networks*, 181:106796, 2025.

[Lu *et al.*, 2023] Yuxiang Lu, Suizhi Huang, Yuwen Yang, Shalayiding Sirejiding, Yue Ding, and Hongtao Lu. Towards hetero-client federated multi-task learning. *arXiv preprint arXiv:2311.13250*, 2023.

[McMahan *et al.*, 2017] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.

[Muhamed *et al.*, 2024] Aashiq Muhamed, Meher Mankikar, and Virginia Smith. Fed up with complexity: Simplifying many-task federated learning with NTKFedavg. In *Privacy Regulation and Protection in Machine Learning*, 2024.

[Netzer *et al.*, 2011] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011.

[Nguyen *et al.*, 2024] Duy Phuong Nguyen, J Pablo Munoz, and Ali Jannesari. Flora: Enhancing vision-language models with parameter-efficient federated learning. *arXiv preprint arXiv:2404.15182*, 2024.

[Ortiz-Jimenez *et al.*, 2023] Guillermo Ortiz-Jimenez, Alessandro Favero, and Pascal Frossard. Task arithmetic in the tangent space: Improved editing of pre-trained models, 2023.

[Ping *et al.*, 2024] Siqi Ping, Yuzhu Mao, Yang Liu, Xiao-Ping Zhang, and Wenbo Ding. Fl-tac: Enhanced fine-tuning in federated learning via low-rank, task-specific adapter clustering. *arXiv preprint arXiv:2404.15384*, 2024.

[Stallkamp *et al.*, 2011] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: A multi-class classification competition. In *The 2011 International Joint Conference on Neural Networks*, pages 1453–1460, 2011.

[Sun *et al.*, 2024] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. Improving lora in privacy-preserving federated learning, 2024.

[Tenison *et al.*, 2023] Irene Tenison, Sai Aravind Sreeramadas, Vaikkunth Mugunthan, Edouard Oyallon, Irina Rish, and Eugene Belilovsky. Gradient masked averaging for federated learning, 2023.

[Tsouvalas *et al.*, 2023] Vasileios Tsouvalas, Yuki Asano, and Aaqib Saeed. Federated fine-tuning of foundation models via probabilistic masking, 2023.

[Vu *et al.*, 2022] Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou, and Daniel Cer. Spot: Better frozen model adaptation through soft prompt transfer, 2022.

[Xiao *et al.*, 2010] J. Xiao, J. Hays, K. A. Ehinger, A. Oliva, and A. Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 3485–3492, June 2010.

[Yadav *et al.*, 2023a] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models, 2023.

[Yadav *et al.*, 2023b] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models, 2023.

[Yang *et al.*, 2024] Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning, 2024.

[Yi *et al.*, 2023] Liping Yi, Han Yu, Gang Wang, and Xiaoguang Liu. Fedlora: Model-heterogeneous personalized federated learning with lora tuning. *arXiv preprint arXiv:2310.13283*, 2023.

[Yu *et al.*, 2024] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch, 2024.

[Zamir *et al.*, 2018] Amir Zamir, Alexander Sax, William Shen, Leonidas Guibas, Jitendra Malik, and Silvio Savarese. Taskonomy: Disentangling task transfer learning, 2018.

[Zhuang *et al.*, 2023] Weiming Zhuang, Yonggang Wen, Lingjuan Lyu, and Shuai Zhang. Mas: Towards resource-efficient federated multiple-task learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 23414–23424, 2023.